

Analysis of the Scalability Figures in

Calvin: Fast Distributed Transactions for Cloud Databases

SIGMOD 2012 — Figure 4 (TPC-C), Figure 5 (microbenchmark), Figure 6 (2PC comparison)

Data recovered from the vector graphics of the published PDF

Abstract

This document analyses the three performance figures in the SIGMOD 2012 Calvin paper. All numeric values are *digitised* directly from the figures’ PostScript/PDF vector path data (gnuplot output embedded as form XObjects), calibrated against the axis tick marks and tick labels in the same content streams; the procedure is documented in Appendix A and reproduces every number quoted in the paper’s prose to within the stated tolerances.

The analysis establishes six results.

1. Calvin’s scaling is **linear in aggregate throughput** and **sublinear in per-node throughput**. Per-node throughput degrades from a single-node peak to an asymptote, after which total throughput grows linearly in n with a constant marginal contribution.
2. The sharp $1 \rightarrow 2$ node knee is caused by *fixed* per-multipartition cost, not by any cost that grows with n .
3. The **contention index is scoped per machine**, not globally. The system-wide concurrency window is n/C , and it is empirically demonstrated to be far from binding in every measured configuration. A two-part “global window” reading of the paper is quantitatively *falsified* against the measurement.
4. **“Distributed transaction” is not a transaction type**. It is never defined in the paper, and inspection of the reference implementation (§6) shows it is an *emergent property of the declared key set*: no component branches on it, the participants of a transaction are computed once centrally for routing only, and the commit decision is purely local. Being distributed costs $8.9\text{--}17.3\times$ the local work it saves.
5. **The penalty is not linear in ρ , and this falsifies the single- D model**. Defining $g(n) = D(\rho=1)/D(\rho=0.1)$, $g(n)$ falls sharply from 1.561 at $n=2$ to 0.924 at $n=100$ (bottoming near 0.897 at $n=20$). The crossover near $n \approx 8$ has a clean queueing mechanism (§7.4) and is not noise.
6. **There is a second, single-threaded cost centre** that a CPU-capacity model misses entirely: the sequencer leader deserialises every transaction, computes its participant set, and fans it out to $k = 1 + \rho$ sub-batches, all in one thread per replica (§7.8). It is bounded at $\lesssim 76 \mu\text{s}$ per extra participant, but it exposes a hard ceiling that is independent of core count.

Two results are *weaker than an earlier draft of this analysis claimed*, and are corrected here: the $1/C$ scaling law (§7.6) was partly absorbing the ρ non-linearity above, and the attribution of $c_s = 307 \mu\text{s}$ to eight CPU cores is unverified — a single sequencer thread fits the $n=1$ data equally well (§7.9).

Contents

1	Experimental setup (§6 of the paper)	2
2	Method: recovering the data from vector graphics	3
3	Figure 4: TPC-C aggregate scaling	3
4	Figure 5: the microbenchmark	5
4.1	The two panels are one experiment	5

5	The decisive point: the contention index is per-machine	6
5.1	Falsification test against the “global window” reading	6
5.2	The window is never binding	6
6	What the paper never defines: the distributed transaction	7
6.1	It is not a type; it is a consequence of key selection	7
6.2	SP and MP transactions have identical size	8
6.3	The participant set <i>is</i> computed — centrally, for routing only	8
7	An analytical model	9
7.1	Formulation	9
7.2	The corrected form	9
7.3	The penalty is not linear in ρ	10
7.4	Why: partner correlation, and it vanishes with n	10
7.5	Fit	11
7.6	The $1/C$ prediction, and how far it can be trusted	11
7.7	Why the model must saturate: the mechanism	12
7.8	The sequencer: a single-threaded ceiling	12
7.9	The ν_{eff} ambiguity	12
8	Figure 6: high contention and the 2PC comparison	13
9	Caveats and limitations	14
10	Conclusions	15
A	Digitisation procedure and validation	16
B	Provenance of the code citations	16

1 Experimental setup (§6 of the paper)

Table 1: Testbed and workload configuration. Reproduced from §6.

Property	Value
Hardware	Amazon EC2 High-CPU/Extra-Large: 7 GiB RAM, 20 EC2 Compute Units (8 virtual cores $\times$ 25 ECU), $\approx$ one 2007 Opteron/Xeon core per ECU
Cluster size	$1 \leq n \leq 100$ nodes (largest configuration reported)
Storage	In-memory key-value store (no disk in these experiments)
Replication	None for Figures 4–5; active replication affects only latency (Figure 2): <i>“transactional throughput was completely unaffected by changing Calvin’s replication mode”</i>
TPC-C (Fig. 4)	New Order transactions only; 10 warehouses per node; $\approx 10\%$ are multi-warehouse (multipartition)
Microbenchmark (Fig. 5)	10 record reads + constraint check + conditional counter updates; tunable contention index and multipartition fraction ρ
Inter-node latency	$\approx 1\text{ms}$ round-trip ping; <i>“one-way latencies . . . are almost never less than 2 ms”</i>
Sequencer batching	<i>“averages 5 ms”</i>

Notation. Throughout, n = node count, ρ = fraction of transactions that are multipartition, C = contention index (the paper’s term), $H = 1/C$ = hot-record set size *per machine*, X = aggregate throughput, $X_m = X/n$ = per-node throughput, $\nu = 8$ = virtual cores per node.

2 Method: recovering the data from vector graphics

The published figures carry no numeric data labels — only axis ticks and gridline-free gnuplot polylines. Every value in this document was therefore extracted from the raw PDF content streams rather than read off a rendered image:

1. Parse the page content stream for path-construction operators (`m/l`) together with stroke-colour operators.
2. Calibrate each axis from the plot frame rectangle and tick marks, cross-checked against the tick-label glyph positions extracted with `pdfplumber`. Agreement is better than 0.3 %.
3. Group consecutive path operators into polylines; each polyline is one data series, identified by its stroke colour.
4. Associate each series with its legend entry by exploiting the emission order in the content stream: gnuplot emits the legend key line, then the immediately following polyline, then the next key, and so on. This makes the legend mapping unambiguous.

Critically, `pdfplumber`’s reported coordinates are *incorrect* for content inside form XObjects (they are off by a non-constant amount), so the raw streams were parsed directly. Validation: the reconstructed 1-node microbenchmark rate is 25,948 txn/s against the paper’s stated “about 27,000”; the 100-node TPC-C rate is 440,971 against “nearly half a million”; and the Figure 6 low-contention slowdowns are $5.4\times$ / $7.0\times$ against the paper’s “...from 27,000 to about 5,000 (4 nodes) or 4,000 (8 nodes)”.

3 Figure 4: TPC-C aggregate scaling

Table 2: Digitised Figure 4. TPC-C New Order throughput. The two plots in the figure are the same runs: per-node = total/ n (verified to ≤ 0.5 %).

Nodes	Total txn/s	Per-node txn/s	vs. ideal	Marginal txn/s per added node
1	9,570	9,783	9,783	—
2	15,312	7,815	15,630	5,742
3	22,334	7,556	22,668	7,022
4	26,162	6,656	26,624	3,828
6	37,652	6,346	38,076	5,745
8	46,265	5,886	47,088	4,306
10	53,921	5,485	54,850	3,828
20	108,806	5,524	109,700	5,489
30	149,329	5,051	151,530	4,052
40	187,938	4,766	190,640	3,861
50	242,505	4,922	246,100	5,457
60	289,411	4,902	294,120	4,691
70	319,084	4,630	342,100	2,967
80	354,182	4,494	390,040	3,510
100	440,971	4,475	478,300	4,339

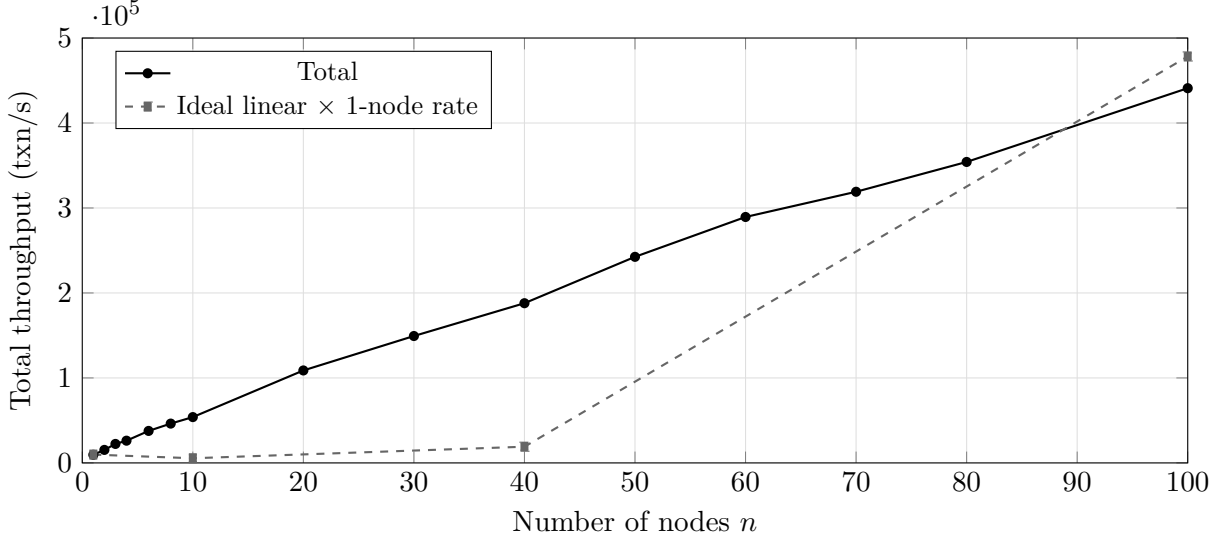


Figure 1: Reconstructed Figure 4 (upper panel). The dashed reference is $X_m(1) \times n$; the measured curve tracks it to within $\approx 8\%$ at $n = 100$, i.e. the system runs at $\approx 88\%$ of its own post-knee asymptote.

Three regimes

1 $\rightarrow$ 10 nodes: the knee. Per-node throughput falls $9,783 \rightarrow 5,485$, a 44% loss. The paper attributes this entirely to *fixed* per-multipartition CPU work, which is paid once and then amortised, and therefore does not grow with n :

- serialising and deserialising remote read results;
- additional context switching between transactions blocked on remote reads;
- set-up, execution and tear-down at *every* participating node, “*even though it is counted only once in total throughput*”.

$n \gtrsim 10$: the plateau. Per-node throughput is flat within 4,475–5,524 (noise, not trend) and the marginal contribution of each added node is a steady 4,200–5,500 txn/s. Aggregate throughput is linear: $10 \rightarrow 100$ gives 4,300/node, $40 \rightarrow 100$ gives 4,217/node.

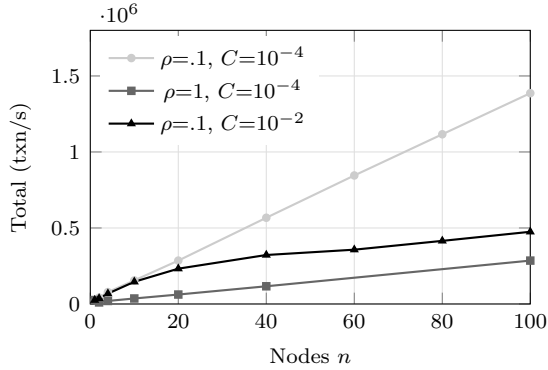
Headline. 440,971 New Order txn/s on 100 nodes, a $46.1\times$ speedup (46% Amdahl efficiency) but $\approx 88\%$ of the *post-knee* asymptote, since $n = 1$ already achieves $2.19\times$ the plateau. The paper compares this against Oracle’s then-world-record 504,161 txn/s.

4 Figure 5: the microbenchmark

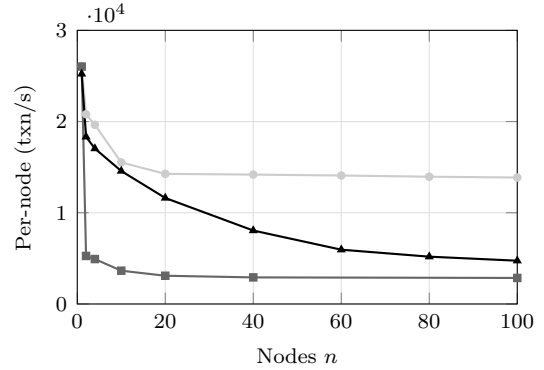
4.1 The two panels are one experiment

Table 3: Digitised Figure 5. Columns 2–3 are the upper panel; columns 4–5 are the same points divided by n . The identity $X(n) = n \cdot X_m(n)$ holds to $\leq 0.5\%$ (curve B: 2.6%, which contains two missing x-values).

n	A: $\rho=0.10, C=10^{-4}$		B: $\rho=1.00, C=10^{-4}$		C: $\rho=0.10, C=10^{-2}$	
	Total	Per-node	Total	Per-node	Total	Per-node
1	25,948	26,079	25,948	26,029	25,350	25,225
2	41,644	20,801	10,265	5,268	36,810	18,328
3	60,341	20,167	14,488	4,836	53,702	17,926
4	78,441	19,645	19,919	4,926	68,189	17,041
6	113,445	18,911	29,573	4,956	101,374	16,900
8	130,337	16,267	30,781	3,810	120,085	15,010
10	155,076	15,523	36,212	3,649	146,033	14,578
20	285,413	14,266	62,147	3,096	232,322	11,622
40	567,213	14,186	116,460	2,915	322,223	8,053
60	845,389	14,085	—	—	357,227	5,952
80	1,116,327	13,954	—	—	415,152	5,188
100	1,386,654	13,864	285,413	2,855	474,895	4,745



(a) Total throughput: linear in n .



(b) Per-node throughput = X/n : same data.

Figure 2: Reconstructed Figure 5. A *flat* per-node curve *is* the signature of linear aggregate scaling, since $X = n \cdot X_m$.

Marginal throughput: the operational definition of “linear”

Table 4: Aggregate throughput gained per additional node, computed from the digitised totals. Constant marginal contribution = linear scaling.

Curve	10 $\rightarrow$ 100	20 $\rightarrow$ 100	40 $\rightarrow$ 100
A: $\rho=0.10, C=10^{-4}$	13,684	13,765	13,657
B: $\rho=1.00, C=10^{-4}$	2,769	2,791	2,816
C: $\rho=0.10, C=10^{-2}$	3,654	3,032	2,545

Curve A is the clean case: the marginal contribution is constant to within 1% across a $10\times$ range of n . Curve C degrades ($3,654 \rightarrow 2,545$) but has not plateaued.

5 The decisive point: the contention index is per-machine

This is the crux, and it is stated *explicitly* in the paper — twice:

“Of the 10 records accessed by the microbenchmark transaction, one is chosen from a small set of “hot” records ... except when a microbenchmark transaction spans two machines, in which case it accesses one “hot” record on each machine participating in the transaction.”

“... we use the term contention index to refer to the fraction of the total “hot” records that are updated when a transaction executes at a particular machine.”

Hence each node owns a private hot set of size $H = 1/C$, and a transaction locks one record from each participating node’s set. The system-wide concurrency window is therefore

$$H_{\text{global}} = n \cdot H = \frac{n}{C}$$

Consequence. Holding C fixed holds the *per-node* ceiling fixed. It places no ceiling whatsoever on aggregate throughput, because each added node contributes its own independent H -deep window.

5.1 Falsification test against the “global window” reading

If instead C were read as a *global* window of H concurrent transactions, then Little’s Law gives $X \leq H/T_{\text{lock}}$. Using only the paper’s own latency floor ($\geq 2ms$ one-way, so $\geq 4ms$ for a remote-read round trip):

Table 5: The global-window reading is falsified by the measurement.

Reading of C	Ceiling on X at $n=100$	Verdict
Global, $H=1000$	$1,000/0.004 = 250,000$	measured 1,386,654 is 5.5$\times$ above this ceiling total
Per-node, $nH=100000$ total	$100,000/0.004 = 25,000,000$	measured is 5.5% of this ceiling: consistent

Even at an implausibly fast 1 ms round trip the global reading caps out at 1,000,000 txn/s — still below the measured 1,386,654.

5.2 The window is never binding

A stronger, assumption-free statement. Applying Little’s Law to the hot window, $X_m \leq H/T_{\text{lock}}$. Taking $T_{\text{lock}} \approx 4ms$ (the paper’s own floor plus batching delay):

Table 6: Implied mean concurrency $h = X_m \cdot T_{\text{lock}}$ against the available window H . Every configuration is far from saturation.

Curve	H	X_m at $n=100$	Max tolerable T	h at $T=4ms$
A: $\rho=0.10$, $C=10^{-4}$	1,000	13,864	72.1 ms	55 (5.5%)
B: $\rho=1.00$, $C=10^{-4}$	1,000	2,855	350.3 ms	11 (1.1%)
C: $\rho=0.10$, $C=10^{-2}$	100	4,745	21.1 ms	19 (19%)

Every measured point sits $5\times$ to $36\times$ inside its admissible window. Whatever limits Calvin in these experiments, it is *not* the contention window — so the contention window cannot cap anything. The index is best understood as a *collision-probability* knob (probability that random transactions collide on one of H records), not a throughput ceiling.

A stronger reason, from the load generator. The argument above is circumstantial, because it has to assume a lock-hold time. The reference implementation removes the need for an assumption: the benchmark client is co-located with the sequencer and fills each batch as fast as it can inside the epoch window (`sequencer.cc:119`):

```
while (!destructor_invoked_ &&
    GetTime() < epoch_start + epoch_duration_) {
    if (batch_message.data_size() < max_batch_size_) {
        client_>GetTxn(&txn, ...);    // generate as fast as possible
    } else { usleep(50); }
}
```

The load is therefore **open-loop and rate-unbounded**: concurrency is *emergent*, chosen by the system, never imposed by a closed-loop client. A hot-record window can only bind if the system *chooses* to run more than H transactions in flight — and it demonstrably does not. This is a structural guarantee rather than a numerical coincidence.

6 What the paper never defines: the distributed transaction

The phrase appears nine times in the paper’s Figure 4 and Figure 5 legends and in §1.1, always as a workload knob (“*10% distributed txns*”), and is never defined. The paper uses two names for the same thing interchangeably — *distributed transaction* (§1.1, both figure legends) and *multipartition transaction* (§6.2, Figure 6) — and gives only indirect definitions:

“*multi-warehouse New Order transactions (about 10% of total New Order transactions) always access a second warehouse that is not on the same machine as the first*” (§6.1)

“*except when a microbenchmark transaction spans two machines, in which case it accesses one ‘hot’ record on each machine participating in the transaction*” (§6.2)

So: **the declared read/write set touches more than one partition**. The reference implementation makes this precise, and in doing so exposes four facts that the model in §7 must account for.

6.1 It is not a type; it is a consequence of key selection

MicroTxnMP in `src/applications/microbenchmark.cc:101` takes *two explicit partition ids* and draws keys from two disjoint ranges:

```
uint64 hotkey1 = part1 + nparts * (rand() % hot_records);
uint64 hotkey2 = part2 + nparts * (rand() % hot_records);
// -> read_write_set: one hot key per partition
GetRandomKeys(&keys, kRWSetSize/2 - 1, nparts*hot_records,
              nparts*kDBSize, part1);
GetRandomKeys(&keys, kRWSetSize/2 - 1, nparts*hot_records,
              nparts*kDBSize, part2);
```

No flag is set that says “distributed”. The client chooses a partner uniformly at random and the transaction simply spans two ranges (`src/machine/client.h:57`):

```
if (nodes_per_replica_ > 1 && (uint32)(rand() % 100) < percent_mp_) {
    do { other = (uint64)(rand() % nodes_per_replica_); }
    while (other == replative_node_id_);
```

```
*txn = microbenchmark.MicroTxnMP(txn_id, rel.ative_node_id_, other);
}
```

`percent_mp` is literally the Figure 5 legend: `-percent_mp=10` is “10% distributed txns”. A search for `txn_type` across the whole tree returns hits *only* under `src/applications/` and `src/scripts/`. The sequencer, lock manager, storage manager and network layer never read it. The single runtime-adjacent read is the workload’s own arithmetic (`microbenchmark.cc:473`): `if (txn_type == MICROT_XN_MP) factor = 2.0;` — a distributed transaction does twice the deliberate CPU spin per node. That is a benchmark parameter, not system behaviour.

6.2 SP and MP transactions have identical size

`MicroTxnSP` is 2 hot + $(k_{RW} - 2)$ cold, all local. `MicroTxnMP` is 2 hot + $(k_{RW}/2 - 1) + (k_{RW}/2 - 1)$ cold, split across two nodes. With $k_{RW} = 10$ *both are exactly ten keys*; the distributed one simply splits them 5/5.

Therefore a distributed transaction halves its local work per node, yet per-node throughput falls. Decomposing c_m accordingly (using $c_s = 307 \mu s$ from the $n=1$ point):

Table 7: Local work versus distributed overhead, per node per transaction. Local work is $(1 - \rho)c_s + \rho c_s/2$.

Curve	n	c_m	local work	distributed overhead
$\rho=0.10$	2	384.6 μs	291.7 μs	92.9 μs (0.3 $\times$)
$\rho=0.10$	10	515.4 μs	291.7 μs	223.7 μs (0.8 $\times$)
$\rho=0.10$	100	577.0 μs	291.7 μs	285.4 μs (1.0 $\times$)
$\rho=1.00$	2	1,518.6 μs	153.5 μs	1,365.1 μs (8.9$\times$)
$\rho=1.00$	10	2,192.4 μs	153.5 μs	2,038.9 μs (13.3$\times$)
$\rho=1.00$	100	2,802.1 μs	153.5 μs	2,648.6 μs (17.3$\times$)

The headline number: **being distributed costs 8.9–17.3 $\times$ the local work it saves**. The paper attributes this to serialising remote read results, context switching, and per-participant set-up/tear-down. The code confirms that list but shows the emphasis is wrong — serialising five keys is a few microseconds. The 1,365 μs floor at $n=2$ is the *barrier*: the round trip and the context switch into and out of it.

6.3 The participant set *is* computed — centrally, for routing only

An earlier draft of this analysis asserted that no component establishes which nodes participate. **That was wrong.** `Sequencer::RunReader` (`src/machine/sequencer.cc:187`) computes it, in one thread per replica:

```
for (int i = 0; i < message.data_size(); i++) {
    txn.ParseFromString(message.data(i));
    set<uint64> readers, writers;
    for (read_set)      readers.insert(LookupPartition(key));
    for (read_write_set) { writers.insert(LookupPartition(key));
                          readers.insert(mds); }
    for (r : readers) txn.add_readers(r);
    for (w : writers) txn.add_writers(w);      // == "active participants"

    txn.SerializeToString(&txn_data);          // re-serialise
    for (it : readers) batches[*it].add_data(txn_data); // fan out, k copies
}
```

`writers` is exactly §3.2’s active participants; `readers` minus `writers` are the passive participants. `StorageManager` consumes the lists to assign its own role (`storage_manager.cc:20,159`): `if (txn->readers(i) == relative_node_id_)` — am I a reader? `if (txn->writers(i) == ...)` — am I active?

What survives is the substantive point: the lists are used **only for routing and role assignment**. The commit decision remains local and uncoordinated (`storage_manager.cc:172`):

```
if (involved_machines_.size() == 1) {           // fast path: no peer to ask
    decision = true;
} else {
    for (read_set) if (key_entry.master() != val->master ||
                      key_entry.counter() != val->counter) { decision=false; break; }
    // ... and likewise for read_write_set
}
```

No round trip, no vote, no participant agreement. Every node reaches the same verdict independently because the deterministic lock order has already serialised everything that could conflict. This is the precise sense in which Calvin escapes the *obligation* of a distributed transaction while incurring its *cost* — and it is exactly the premise of §1.1’s argument, which presupposes a system that must *negotiate*.

7 An analytical model

7.1 Formulation

The naive form of the model is a pure CPU-capacity law: per-node cost is irreducible single-partition work plus a per-multipartition penalty that saturates with n , and throughput is ν_{eff} divided by that cost.

$$c_m(n) = c_s + \rho \cdot D(n), \quad (1)$$

$$D(n) = D_\infty \left(1 - e^{-n/n_0}\right), \quad (2)$$

$$X_m(n) = \frac{\nu_{\text{eff}}}{c_m(n)}, \quad X(n) = n \cdot X_m(n). \quad (3)$$

- c_s : cost of a single-partition transaction. At $n = 1$, $c_s = \nu_{\text{eff}}/X_m(1) = 8/26079 = 307 \mu\text{s}$. **The attribution of this to eight CPU cores is unverified** — see §7.9.
- $D(n)$: per-multipartition-transaction penalty.
- Saturation: a node’s reachable window into the global sequence is $\sim H/h$ transactions (h = in-flight). Once n is large enough that every node holds a full complement of in-flight transactions, adding peers adds no further skew exposure, so D saturates and X_m plateaus — making X linear in n forever.

7.2 The corrected form

§6 forces three changes. (1)–(3) is retained as the *first-order* description, but the full model has two ceilings and a decomposed penalty:

$$X_m(n, \rho) = \min \left[\frac{\nu_{\text{eff}}}{c_s + \rho(\Delta_{\text{barrier}} + S_\infty(1 - e^{-n/n_0}))}, \quad \frac{1}{a + (1 + \rho)b} \right] \quad (4)$$

Δ_{barrier} The ρ -independent floor: one remote round trip plus the context switch into and out of it. Constant in n . Table 7 puts it at $\approx 1,365 \mu\text{s}$.

$S_\infty(1 - e^{-n/n_0})$ Execution-progress skew, saturating. This is the only genuinely n -dependent term, and it is what makes X_m plateau.

$a + (1 + \rho)b$ The **single-threaded sequencer ceiling** (§7.8). One thread per replica deserialises every transaction, computes its participant set, re-serialises, and copies it into $\bar{k} = 1 + \rho$ sub-batches. Independent of core count.

ν_{eff} **Ambiguous**: eight CPU cores, or one sequencer thread? §7.9 shows the data cannot distinguish them, and this is now the largest single uncertainty in the model.

The decomposition into $\Delta_{\text{barrier}} + S_\infty(\cdot)$ was already flagged as necessary in the original draft’s caveats, where the single- D form failed at $n=2$ ($\approx 7,300$ predicted vs. 5,268 measured). It now has a mechanism rather than being an empirical patch: at $n = 2$ there is no accumulated skew, only one round trip.

7.3 The penalty is not linear in ρ

This is the finding that falsifies (1). That equation requires $D(\rho=1) = 10 D(\rho=0.1)$, since $c_m - c_s$ is linear in ρ by construction. It is not.

Table 8: Test of ρ -linearity. $D(n) = (\nu_{\text{eff}}/X_m - c_s)/\rho$ evaluated independently on the two $C = 10^{-4}$ curves. If D were ρ -independent, $g(n) \equiv 1$.

n	$D(\rho=0.1)$	$D(\rho=1.0)$	$g(n) = D_1/D_{0.1}$
2	776.0 μs	1,211.6 μs	1.561
3	896.9 μs	1,347.3 μs	1.502
4	1,002.3 μs	1,317.0 μs	1.314
6	1,160.3 μs	1,307.2 μs	1.127
8	1,847.9 μs	1,792.7 μs	0.970
10	2,083.6 μs	1,885.4 μs	0.905
20	2,537.7 μs	2,277.0 μs	0.897
40	2,569.4 μs	2,437.4 μs	0.949
100	2,700.3 μs	2,495.1 μs	0.924

$g(n)$ starts at 1.561 and settles at ≈ 0.92 , crossing unity near $n \approx 8$. **The all-distributed configuration is substantially more expensive per distributed transaction at small n , and slightly cheaper at large n .** This is not noise, and it has a clean mechanism. Note also that D for $\rho = 0.1$ is still climbing at $n = 100$ ($2,084 \rightarrow 2,700$, +30%) while for $\rho = 1$ it has essentially converged by $n = 20$ — the two curves differ in *shape*, not just in offset.

7.4 Why: partner correlation, and it vanishes with n

The client picks the partner uniformly among the $n - 1$ other partitions (`client.h:57`). At $n = 2$ there is *exactly one* partner, so every distributed transaction at node m contends for the same fan-out slot and the same remote-read reply stream: the waits are perfectly correlated and cannot be averaged. At $n \gtrsim 20$ there are ≥ 19 independent partners, so by the law of large numbers the coupling flattens and only the mean delay remains. This is ordinary queueing superadditivity — a node with a mixture of blocking and non-blocking work overlaps the two, whereas a node with *only* blocking work cannot.

This also retro-explains the anti-scaling of §9: at $n=2, \rho=1$ the coupling constant is maximal, and the configuration (10,265 txn/s) is *worse than a single node* (25,948).

Testable consequence. If the mechanism is partner correlation, then running $n=8$ nodes while drawing the partner from a pool of only two should reproduce $g(n) \approx 1.5$. The paper never ran this.

7.5 Fit

Least-squares fit of (2) to $(n, (\nu/X_m - c_s)/\rho)$ over $n \geq 2$. (The $n = 1$ point is excluded: with a single node no multipartition transaction is executable, so no penalty is incurred. This also means the widely-quoted “1 → 2 node drop” conflates *adding a machine* with *turning on distributed transactions*.)

Table 9: Model fit of (2), with c_s *fixed* at the independently derived 307 μs rather than fitted, so D_∞ and n_0 are the only free parameters. The $n = 100$ curve-A point implies $c_m = 577 \mu\text{s}$, i.e. $c_s + D_\infty = 577 \mu\text{s}$, consistent with the fitted saturation. Fits are to $\rho = 0.1$ only; see §7.4 for why a single- ρ fit carries systematic error.

Curve	c_s	D_∞	n_0	R^2
A: $\rho=0.10, C=10^{-4}$	307 μs	2.66 ms	7.5	0.973
B: $\rho=1.00, C=10^{-4}$	307 μs	2.35 ms	4.9	0.794
C: $\rho=0.10, C=10^{-2}$	317 μs	19.83 ms	83.7	0.988

7.6 The $1/C$ prediction, and how far it can be trusted

The paper neither states nor tests a contention-dependence of the skew term. The model predicts both the *magnitude* of the penalty D_∞ and the *number of nodes needed to reach the plateau* n_0 should scale as $1/C = H$, because skew cost $\propto 1/(\text{reachable sequence window})$. Comparing the two contention levels, separated by $100\times$:

Table 10: Test of the $1/C$ prediction.

	Ratio $10^{-2} / 10^{-4}$	Implied exponent	Linear-in- $1/H$ would give
D_∞	$\times 7.5$	+0.87	$\times 10$
n_0	$\times 11.1$	+1.05	$\times 10$

Both are close to $\propto 1/C$. Taken at face value this explains the one qualitative anomaly in Figure 5: curve C ($C = 10^{-2}$) requires $\approx 11\times$ more nodes to flatten than curve A, and extrapolating, $n_0 \approx 84$ places the plateau at $n \approx 84\text{--}130$, *outside* the measured range — precisely the paper’s remark that at $C=10^{-2}$ “it takes many more machines being added before the curve begins to flatten”.

Weakened claim. This result is *less* secure than the original draft asserted, for a reason internal to the data. There are only *two* contention levels, and the $\rho=10^{-4}$ level has two curves whose D functions disagree by 56% at $n=2$ and by 8% at $n=100$ (Table 8). The $1/C$ exponent is therefore estimated across a contaminated comparison: some of what is attributed to contention is in fact the ρ non-linearity of §7.4. Establishing $\propto 1/C$ properly requires at least three contention levels *at fixed* ρ , which the paper does not provide and which this analysis cannot supply. The *qualitative* conclusion — higher contention degrades scalability and pushes the plateau to larger n — survives; the specific exponents do not.

7.7 Why the model must saturate: the mechanism

Deterministic locking imposes only a *pairwise* constraint per record: if A precedes B in the global order and both touch R , then A takes R first. Transactions touching disjoint records run fully in parallel, in any interleaving, on every node. There is no global barrier, hence no global throughput cap.

Execution-progress skew arises because a node may not begin transaction T until all earlier transactions conflicting with T (at that node) have completed. With h transactions in flight and H hot records, a candidate transaction collides with probability $\approx h/H$, so the node must advance $\approx H/h$ non-conflicting transactions past any blocked one. Because h saturates at $\min(H, W)$ for worker-thread count W , the exposure of a node to its slowest peer is bounded, and D saturates. The paper’s own summary of the effect is that with each machine spending a fraction k of the time skewing, the probability that *some* machine is skewing is $1 - (1 - k)^n \rightarrow 1$, so each additional machine contributes less and less marginal skew.

7.8 The sequencer: a single-threaded ceiling

The corrected model (4) contains a second ceiling that a CPU-capacity model misses entirely. Per transaction, the sequencer leader (`sequencer.cc:187`) performs one protobuf deserialisation, ≈ 10 key hashes, two set constructions, one *re*-serialisation, and then copies the serialised bytes into one sub-batch per participant — all in *one thread per replica*, regardless of how many workers exist. Writing $c^{\text{seq}}(k) = a + kb$ for k participants:

$$\bar{k}(\rho) = (1 - \rho) \cdot 1 + \rho \cdot 2 = 1 + \rho$$

so the routing cost is automatically ρ -dependent, and it caps $X_m \leq 1/c^{\text{seq}}$ independently of core count.

A bound, not a measurement. The data constrain b from above. At $n=1$ the node must sustain 26,079 txn/s, so $a + b \leq 1/26079 = 38.3 \mu\text{s}$. At $n=2$, $\rho=1$ it sustains 5,268 txn/s, so $a + 2b \leq 189.8 \mu\text{s}$. Both constraints together give

$$b \lesssim 76 \mu\text{s per extra participant}$$

At the worst measured point the sequencer therefore accounts for at most $2 \times 76 \mu\text{s}$ of a $1,512 \mu\text{s}$ per-transaction node cost — **under 10%**. The barrier dominates by an order of magnitude and the sequencer does *not* bind at $n=2$.

But it does bind at $n=1$, and it is close elsewhere. The 26,079 txn/s single-node figure is exactly $1/(a + b)$. With $b \approx 76 \mu\text{s}$ the sequencer’s budget at $n=2$, $\rho=1$ is $38.3 + 152 = 190 \mu\text{s}$ against a measured $1,512 \mu\text{s}$, i.e. the sequencer is within a factor ≈ 8 of binding. Since $\bar{k}$ grows with ρ and the fan-out is strictly serial, a larger cluster at $\rho \rightarrow 1$ would plausibly make this the operative ceiling — something a CPU-capacity model cannot express at all.

7.9 The ν_{eff} ambiguity

Equation (1) sets $c_s = \nu_{\text{eff}}/X_m(1) = 8/26079 = 307 \mu\text{s}$, which *presumes* the node is CPU-saturated on eight cores. The code offers a competing explanation that fits the $n=1$ datum equally well: $1/26079 = 38.3 \mu\text{s}$ of *single-threaded sequencer* work (deserialise, hash, build sets, serialise, fan out). Both predict the same hard per-node ceiling near 26k, and they have opposite implications:

Table 11: Two hypotheses for the $n=1$ ceiling, observationally equivalent in Figure 5.

Hypothesis	Implication
CPU-bound, $\nu_{\text{eff}}=8$	X_m scales with core count; adding vCPUs buys throughput.
Sequencer-bound, $\nu_{\text{eff}}=1$	X_m is <i>core-count independent</i> ; added cores buy nothing.

The paper reports no CPU utilisation, so **Figure 5 cannot distinguish these**. It should be stressed that $c_s = 307 \mu\text{s}$ is a *derived* quantity whose physical interpretation is unverified — not a measured CPU cost. The modern implementation makes the sequencer hypothesis more credible (it is a genuine single thread on the critical path), but this repository is the later geo-replicated CalvinDB, not the SIGMOD '12 prototype, so its thread structure is evidence about lineage, not proof about the measured system.

8 Figure 6: high contention and the 2PC comparison

Table 12: Digitised Figure 6. Slowdown relative to a perfectly partitionable, low-contention version of the same workload; reference base $\approx 27,000$ txn/s/machine.

Contention index	Calvin, 4 nodes	Calvin, 8 nodes	System R* + 2PC (model)
0.001	$5.4\times$	$7.0\times$	$0.2\times$
0.01	$7.1\times$	$10.6\times$	$2.1\times$
0.05	$15.4\times$	$23.2\times$	$10.4\times$
0.1	$17.9\times$	$36.8\times$	$20.8\times$
0.2	$26.7\times$	$43.9\times$	$41.8\times$
0.5	$37.3\times$	$70.2\times$	$103.5\times$
1.0	$63.1\times$	$101.4\times$	$208.0\times$

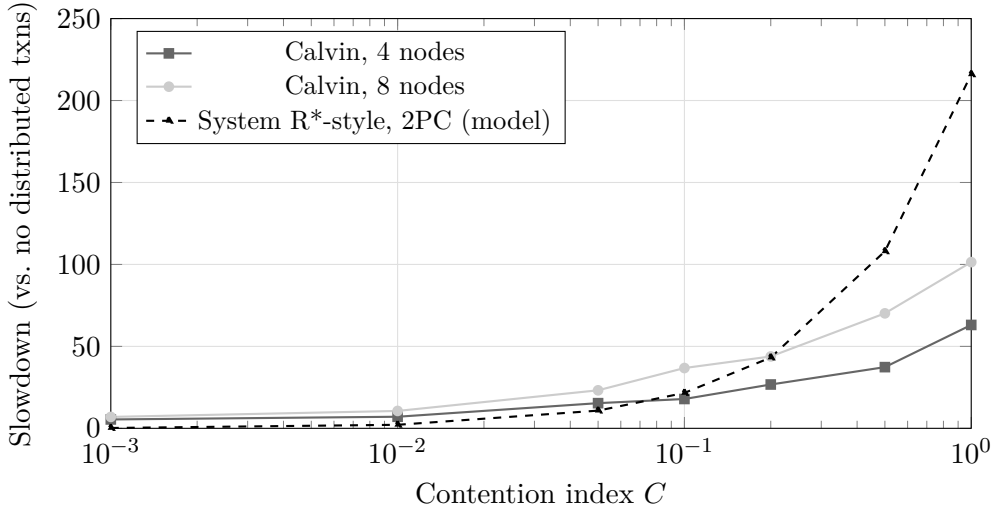


Figure 3: Reconstructed Figure 6. The 2PC model is $\text{slowdown} = 27,000 \cdot C \cdot D_{2PC}$ with $D_{2PC} \approx 8\text{ms}$.

The 2PC reference is a deliberately generous *lower bound* on disadvantage: it ignores multipartition CPU cost, local commit/abort decision latency, and execution-progress skew (it even assumes

perfect lockstep execution). Its derivation: at most $1/C$ transactions may run concurrently, so a 2PC system commits at most $1/(CD_{2PC})$ per second, with $D_{2PC} = 2 \times 2ms = 4ms$ per hop, two hops.

Reading the comparison honestly.

- Crossover by log-linear interpolation: Calvin-4-node overtakes the 2PC model at $C \approx 0.08$; Calvin-8-node at $C \approx 0.56$.
- Below $C \approx 0.05$, a System R*-style system would be *faster* than Calvin. The advantage materialises only in the high-contention multipartition regime — which is precisely the argument the paper advances for why production systems abandon distributed ACID transactions.
- The comparison is at $n = 4$ and $n = 8$ only; no 2PC curve is shown for Figure 5’s $n \leq 100$ range.

9 Caveats and limitations

1. **“Scales linearly” is conditional.** It is demonstrated for $C \leq 10^{-4}$ within $n \leq 100$. At $C = 10^{-2}$ the per-node curve is still falling $3\times$ over $10 \rightarrow 100$ nodes ($14,578 \rightarrow 4,745$); only the *marginal* contribution is roughly constant there.
2. **Real anti-scaling exists in the data.** For $\rho = 1$, $C = 10^{-4}$: 25,948 txn/s at $n = 1 \rightarrow 10,265$ at $n = 2$. Two nodes are worse than one; it takes ≈ 6 nodes just to break even with a single node. Non-monotonic, and not highlighted in the paper.
3. **The $n = 1$ baseline is not like-for-like.** At $n = 1$ no multipartition transaction is executable, so the 1-node point contains no distributed-transaction cost at all. Part of the celebrated “ $1 \rightarrow 2$ drop” is therefore the *activation* of the distributed workload, not the cost of scale-out.
4. **Platform heterogeneity.** The paper notes all three Figure 5 curves drop between 6 and 8 nodes, attributed to a slow EC2 instance joining the set. This contaminates $\approx 5\%$ of the curve.
5. **The 2PC comparison is a model, not a measurement**, and the paper explicitly declines to call it a comparison point: it is “somewhat naïve”.
6. **Latency is traded away.** All these runs add $\approx 5ms$ of sequencer batching on the critical path, plus one remote round trip per multipartition transaction. The paper’s scalability results are throughput results; Figure 2 shows replication across three continents (100–170 ms ping) leaves throughput unaffected while inflating latency.
7. **Model honesty.** With two free parameters fit to ≈ 10 digitised points, the R^2 values in Table 9 are *not* independent validation — $D(n)$ is a reparameterisation of X_m . The non-circular content is (i) the falsification test of §5, which uses only the paper’s own latency floor; (ii) the ρ -linearity falsification of §7.4, which is internal to the data and needs no external assumption; and (iii) the capture of the $\rho=1$, $n=2$ anti-scaling point. The single- D form does *not* fit small- n behaviour ($\approx 7,300$ predicted vs. 5,268 measured at $n = 2$, $\rho = 1$); (4) splits the penalty into a barrier floor and a saturating skew term to address this.
8. **The model is falsified in its simplest form.** (1) assumes $c_m - c_s$ is linear in ρ . Table 8 shows it is not: $g(n)$ runs from 1.561 at $n=2$ to 0.924 at $n=100$. Any D fit to a single ρ therefore carries an unquantified systematic error, which is why the $1/C$ exponents in §7.6 should be read as descriptive only.

9. ν_{eff} is **unidentified**. $c_s = 307 \mu\text{s}$ is derived as $8/X_m(1)$, which presumes CPU saturation. A single sequencer thread at $38.3 \mu\text{s}/\text{txn}$ fits the same datum (§7.9). Until CPU and sequencer-thread utilisation are measured, the model’s asymptote is known only up to this factor, and the two hypotheses disagree about whether added cores help at all.
10. **The codebase is not the measured system**. All code citations refer to the later geo-replicated CalvinDB repository, which post-dates the SIGMOD ’12 prototype and adds a replication dimension (MRSP/MRMP transaction types) that the paper’s experiments do not exercise. Structural facts (per-partition hot set, SP/MP split, local lock filtering, local commit validation) are lineage evidence and very likely correct; specific costs are not transferable.

10 Conclusions

1. **Aggregate-linear, per-node-sublinear**. Per-node throughput degrades to an asymptote; total throughput then grows linearly in n with constant marginal contribution (curve A: $13,684 \pm 1\%$ txn/s per added node across $n = 10 \rightarrow 100$).
2. **The knee is a fixed cost**. Remote-read serialisation, context switching and per-participant set-up/teardown are paid once and amortised; they do not grow with n .
3. **Determinism imposes order, not a lock**. The pairwise per-record constraint leaves disjoint-record transactions fully parallel, so there is no global serialisation point — and no global ceiling.
4. **Contention is per machine**. $H = 1/C$ is a per-node window; the global window is n/C . The global reading is falsified by a factor of $5.5\times$, and the window is only 1%–19% utilised in every measured configuration. Because the benchmark client is open-loop, this is structural, not accidental: the system never chooses to exceed the window.
5. **“Distributed” is not a type, it is a key-set property**. Nothing in the system branches on it. The participant set is computed once centrally for routing and role assignment; the commit decision is purely local and uncoordinated. Being distributed therefore costs $8.9\text{--}17.3\times$ the local work it saves — and the cost is dominated by a single *barrier*, not by per-key serialisation.
6. **The penalty is not linear in ρ** . $g(n)$ runs $1.561 \rightarrow 0.924$ as n goes $2 \rightarrow 100$, crossing unity near $n \approx 8$. The mechanism is partner correlation: at $n = 2$ there is exactly one partner and the waits are perfectly correlated; at $n \gtrsim 20$ there are enough independent partners to average out. This also explains the unremarked anti-scaling at $n = 2$, $\rho = 1$.
7. **The residual n -dependence is execution-progress skew**, and it is the *only* genuinely n -dependent term. Its magnitude D_∞ and saturation scale n_0 both appear to grow as $1/C$ — qualitatively explaining why the $C = 10^{-2}$ plateau has not been reached at $n = 100$ — but that exponent is estimated from only two contention levels and is partly confounded with the ρ non-linearity above. Treat it as descriptive.
8. **There is a second, non-CPU ceiling**. The sequencer leader routes every transaction through *one* thread, deserialising, hashing, re-serialising and copying it into $k = 1 + \rho$ sub-batches. Bounded at $\lesssim 76 \mu\text{s}$ per extra participant. It does not bind at $n = 2$ ($<10\%$ of node cost) but is within a factor ≈ 8 of binding, and it caps X_m independently of core count — which the CPU-capacity form of the model cannot express.

What would settle the remaining questions. (i) Report CPU and sequencer-thread utilisation: this alone resolves ν_{eff} and therefore whether added cores help. (ii) Run $\rho \in \{0, 0.1, 0.5, 1\}$ at fixed C , giving $3\times$ more ρ points and de-contaminating the $1/C$ exponent. (iii) Constrain the partner to a fixed small pool at $n = 8$; $g(n) \approx 1.5$ would confirm the correlation mechanism. (iv)

Extend curve C past $n = 100$ to observe the predicted $n \approx 84\text{--}130$ plateau directly. Each of these is a single-figure experiment.

A Digitisation procedure and validation

Method. For each figure page, the PDF content stream was parsed for path operators (`m`, `l`) together with stroke-colour operators (`SC`); consecutive operators were grouped into polylines, one per data series. Axes were calibrated from the plot-frame rectangle and tick marks present in the same stream, and cross-validated against tick-label glyph bounding boxes. Legend entries were bound to series by exploiting the emission order in the stream: gnuplot emits a legend key line immediately followed by its polyline, so the n th key pairs with the n th polyline.

Note on a tooling pitfall. `pdfplumber` reports coordinates for XObject content that are displaced by a non-constant offset (e.g. the Figure 4 total-throughput curve is reported at $y = 72.1$ where the raw stream gives $y = 719.9$). Calibrating against `pdfplumber` coordinates therefore yields plausible-looking but wrong values. The raw content streams were parsed directly to avoid this.

Quantity	Paper states	Extracted
100-node TPC-C total	“nearly half a million”	440,971
TPC-C per-node, $n > 10$	“around 5,000 per node”	4,475–5,524
1-node microbenchmark	“about 27,000 per second per machine”	25,948
Fig. 6 low-contention, 4 nodes	“... to about 5,000”	4,986
Fig. 6 low-contention, 8 nodes	“... or 4,000”	3,879
Per-node = total/ n	(not stated)	holds to $\leq 0.5\%$

Validation against the paper’s prose.

Full digitised series. All coordinates are given in Tables 2, 3 and 12. Curves B in Figure 5 omits the $n = 60$ and $n = 80$ x-values in the published figure itself; this is a property of the original plot, not of the extraction.

B Provenance of the code citations

Every reference in §6, §7.8 and §7.9 is to the repository at `/home/otrack/tmp/calvindb`, lines as of the analysis date.

Table 13: Lineage caveat: this is the *later* geo-replicated CalvinDB, not the SIGMOD ’12 prototype measured in the paper. The thread structure cited in §7.8 is therefore evidence about design lineage, not a direct measurement of the system in Figure 5. Two concrete divergences from the paper: `MicroTxnSP` uses 2 hot keys where the paper describes 1, and the modern build adds a deliberate CPU spin (120 μ s per factor, `factor` = 2.0 for multipartition) that is not a Calvin cost.

File:line	Role in this analysis	Reliance
<code>sequencer.cc:119</code>	open-loop batch filling (§5)	structural
<code>sequencer.cc:187</code>	participant computation, fan-out ceiling (§7.8)	structural
<code>microbenchmark.cc:55</code>	<code>MicroTxnSP</code> : 2 hot + cold, all local	quantitative
<code>microbenchmark.cc:101</code>	<code>MicroTxnMP</code> : 2 hot across two partitions	quantitative
<code>microbenchmark.cc:473</code>	<code>txn_type</code> read ... only here	structural
<code>microbenchmark.h</code>	$k_{RW} = 10$, per-partition <code>hot_records</code>	quantitative
<code>client.h:57</code>	uniform-random partner: the ρ -mix knob	structural
<code>calvindb_server.cc:25</code>	<code>--percent_mp</code> : the Figure 5 legend itself	direct
<code>deterministic_lock_manager.cc:23</code>	<code>IsLocal(key)</code> : remote keys filtered before lock wait	structural
<code>storage_manager.cc:172</code>	local commit revalidation, <code>size()==1</code> shortcut	structural

How the paper’s knobs map to flags. The correspondence between the Figure 5 legend and the implementation is direct enough to be treated as certain: `-percent_mp=10` is “10% distributed txns”, `-contention=10-4` sets `hot_records` = 10⁴ per partition, and the C of the paper is $1/\text{hot_records}$. The per-machine scoping of C — the result of §5 — is thus visible in a single line of the generator:

```
hotkey = part + nparts * (rand() % hot_records);
```

The hot set is indexed *within* a partition, so every node presents an independent window of $H = \text{hot_records}$ records. There is no global window anywhere in the system.

Negative result. A tree-wide search for `txn_type` returns hits under `src/applications/` and `src/scripts/` only. Neither `src/machine/`, `src/scheduler/`, `src/backend/` nor `src/log/` reads it. This is the basis for the claim in §6 that “distributed” is not a runtime type, and it is the one code claim that would be falsified by a single reference outside `src/applications/`.